Don't forget that I/O is never certain in an uncertain world—exceptions will be raised on most errors, and you should be ready to rescue them and take appropriate action.

Writing to Files

So far, we've been merrily calling puts and print, passing in any old object and trusting that Ruby will do the right thing (which, of course, it does). But what exactly *is* it doing?

With a couple of exceptions, every object you pass to puts or print is converted to a string by calling that object's to_s method. If for some reason the to_s method doesn't return a valid string, a string is created containing the object's class name and ID, something like <ClassName:0x123456>. This example opens a file for writing (note the mode string is "w", then reads the file in and prints its contents to STDOUT). As with the other Kernel methods, there's an equivalent IO#puts that we can use.

```
File.open("output.txt", "w") do |file|
  file.puts "Hello"
  file.puts "1 + 2 = #{1+2}"
end

puts File.read("output.txt")
```

produces:

```
Hello
1 + 2 = 3
```

There is one slight differences between puts and print. The puts method inserts a newline after the output unless the output already ends in a newline, print does not.

There are two exceptions to the "every object calls its to_s method. The nil object will print as the empty string, and an array passed to puts will be written as if each of its elements in turn were passed separately to puts.

More generally, we have File#write, which writes its argument to the file, converting non-strings to string with to_s. The difference being that write returns the number of bytes written to the file, while puts returns nil.

```
File.open("output.txt", "w") do |file|
  file.write "Hello"
  file.write "1 + 2 = #{1+2}"
end

puts File.read("output.txt")
```

produces:

```
Hello1 + 2 = 3
```

What if you want to write binary data and don't want Ruby messing with it? Well, normally you can simply use print and pass in a string containing the bytes to be written. However, you can get at the low-level input and output routines if you really want—look at the documentation for sysread and syswrite at https://docs.ruby-lang.org/en/master/IO.html#method-i-sysread.

And how do you get the binary data into a string in the first place? The three common ways are to use a literal, poke it in byte by byte, or use Array#pack, which takes an array of data and packs it into a string.

```
str1 = "\001\002\003" # => "\u0001\u0002\u0003"
```